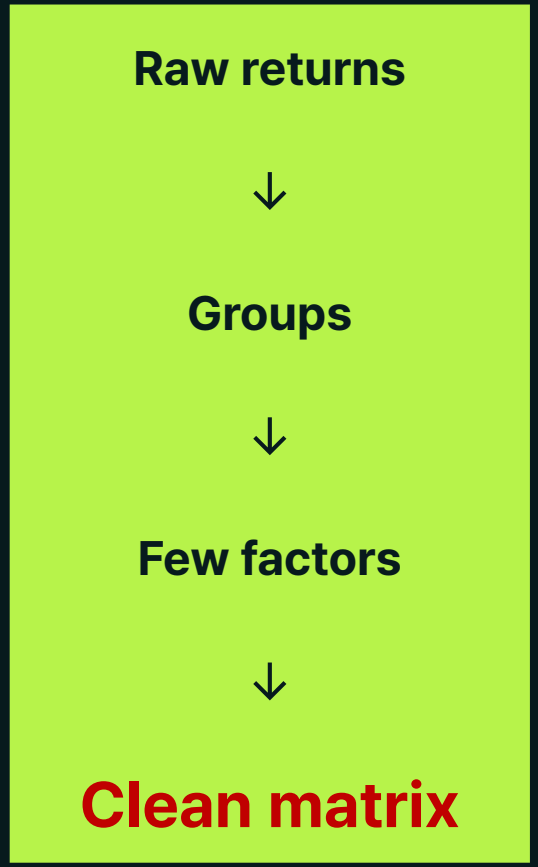
Unsupervised Learning
Group, Compress, Clean

Finding structure when there is no answer key



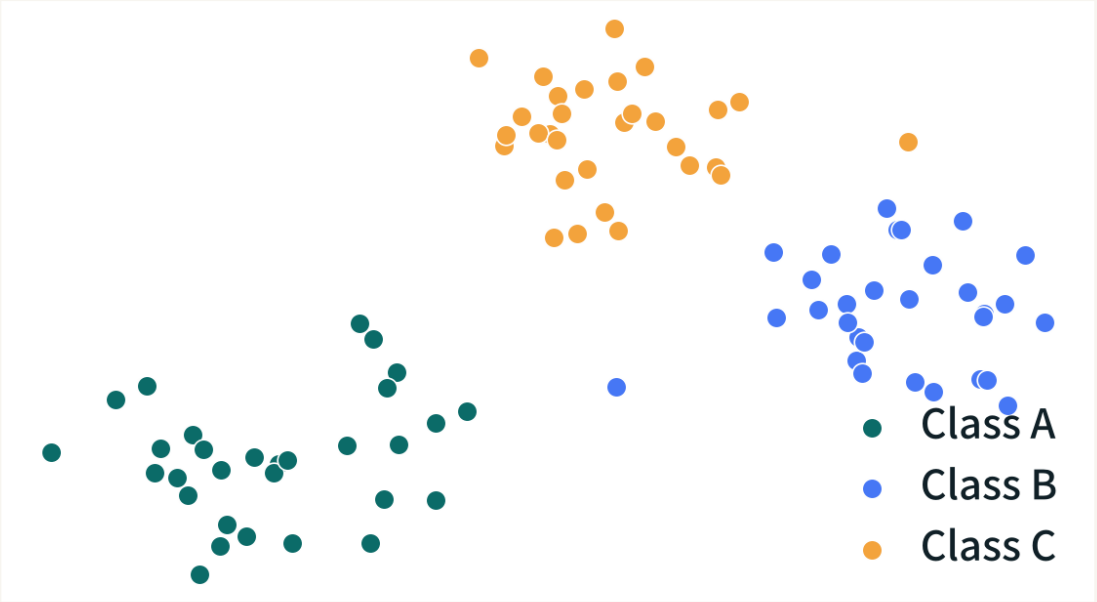
Three jobs and one build

01	**Finding Groups**	**Group the rows** K-Means · Hierarchical · DBSCAN
02	**Shrinking Features**	**Squeeze the columns** Curse of dimensionality · PCA · t-SNE
03	**Cleaning Noise**	**Fix the correlation matrix** Estimation gap · Marchenko-Pastur · Denoising
04	**Build It with Claude Code**	**Three prompts, every figure** Where · Input · Output · Feedback

Unsupervised learning works without an answer key

Same points, two situations: with labels (left) and without labels (right)

Supervised: every point carries its answer



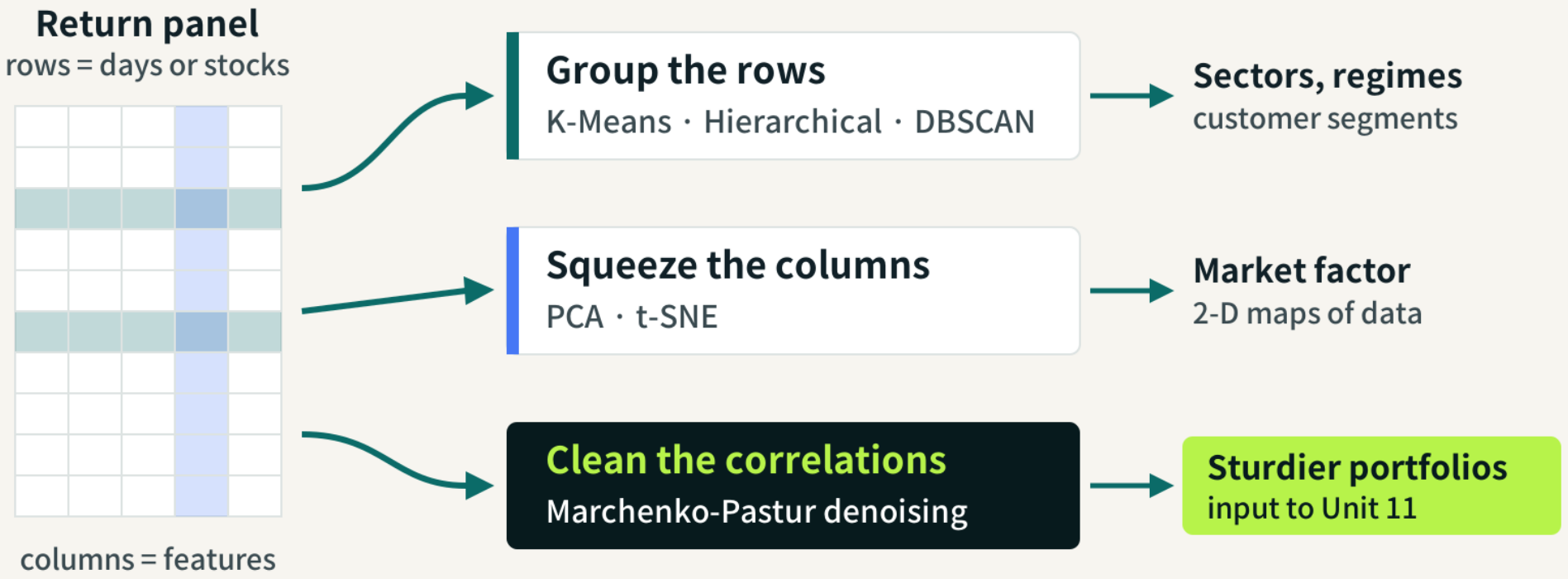
Unsupervised: only the points themselves



※ No accuracy score exists here. A result is judged by whether the structure makes sense and helps.

Each tool works on a different part of the data

Rows are grouped, columns are compressed, and the correlation matrix is cleaned



01

Finding Groups

K-Means · Hierarchical clustering · DBSCAN

K-Means minimises the distance to each group's center

Center (centroid) = the average point of a group; K = number of groups you choose

What it minimises

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

How a center is updated

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

STEP A

Assign

Join the nearest center

STEP B

Update

Move to the group mean

STOP

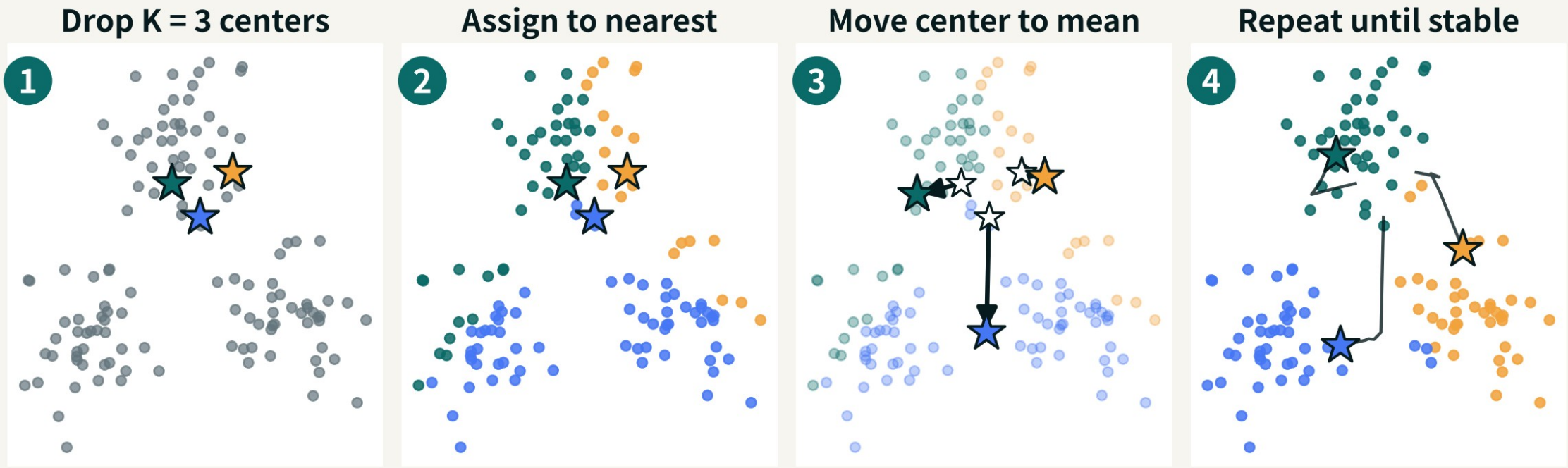
Converged

No point changes group

Both steps can only lower J, so the loop always stops.

K-Means alternates two moves until nothing changes

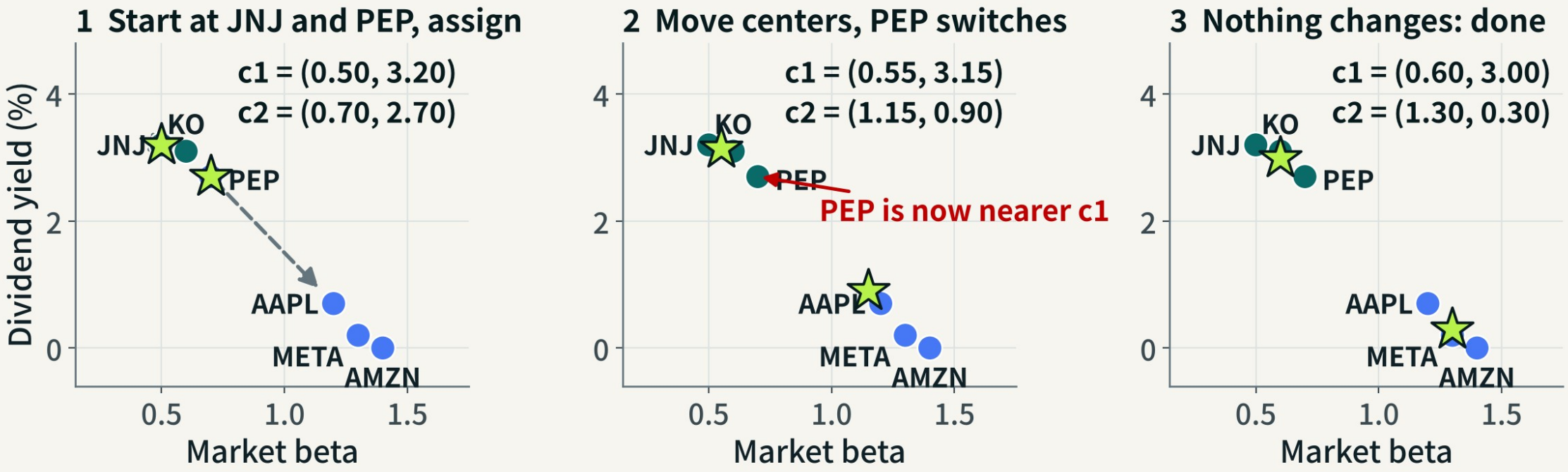
Simulated 2-D data with three hidden groups and a deliberately poor start



※ Panel 4 traces each center's path. Real runs restart 10 times (n_init = 10) and keep the best J.

Six stocks split into defensive and growth in two features

Market beta and dividend yield (%). K = 2, starting centers placed on JNJ and PEP



※ Step 2: PEP's squared distance is 0.23 to c1 but 3.44 to c2, so it switches. Illustrative figures.

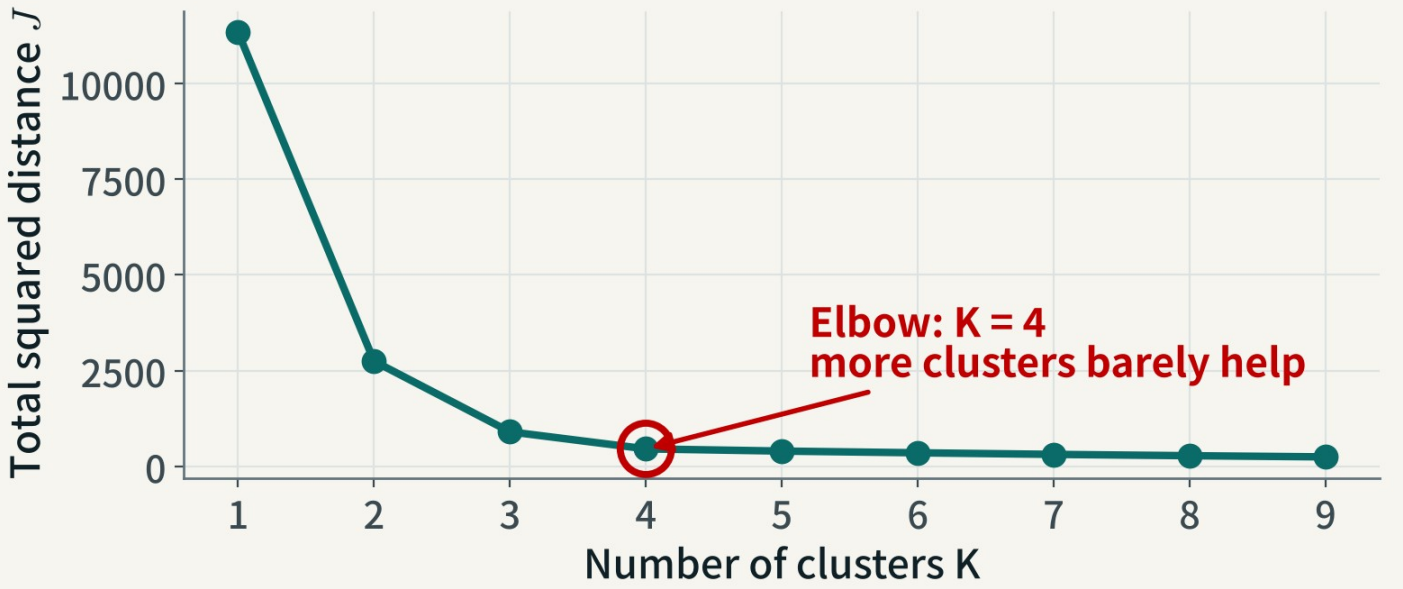
The elbow picks K where extra clusters stop paying

J always falls as K grows, so look for the bend, not the minimum

Reading the curve

- Too few K: lumped
- Too many K: split noise
- Bend: real groups

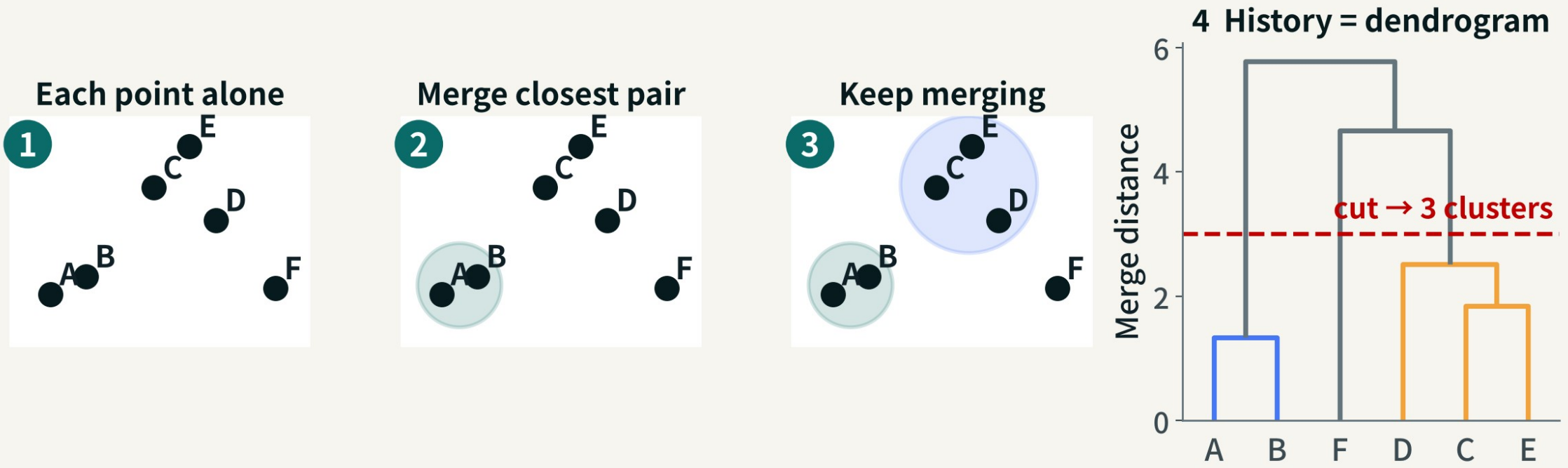
Elbow + common sense



※ Simulated data with four true groups

Hierarchical clustering merges the closest groups step by step

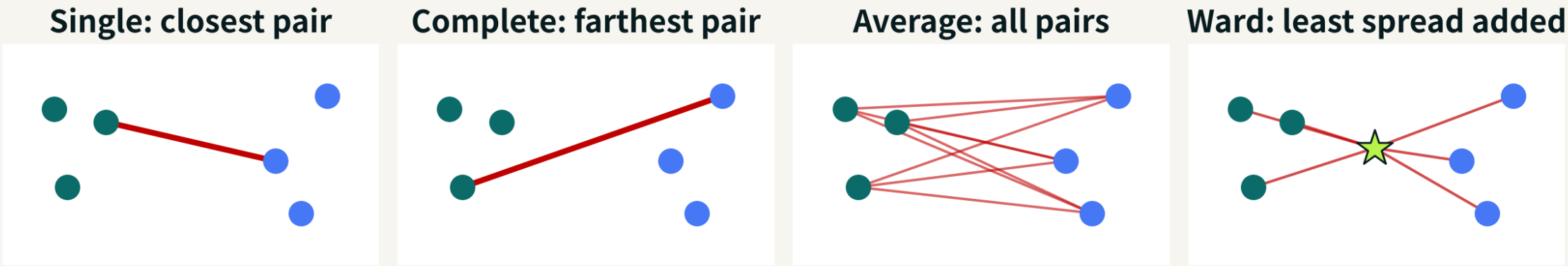
Agglomerative (bottom-up): start with N single points and merge until one group remains



※ The height of each join = the distance at which two groups merged. Cut the tree to choose the number of clusters.

The linkage rule defines the distance between two groups

Points are easy to measure; groups need a rule. The red lines show what each rule measures



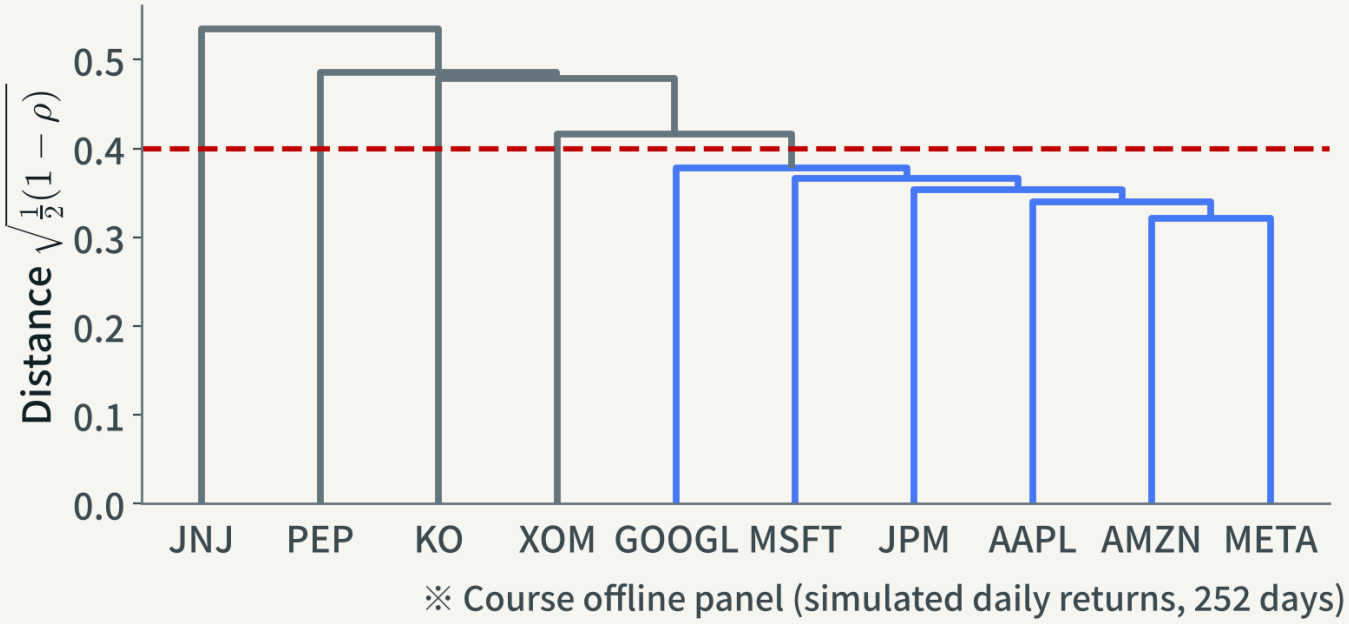
※ Single links long chains, Complete keeps groups compact, Ward is the usual default.

Correlation distance turns a return panel into a stock family tree

Stocks that move together sit close; López de Prado (2016) uses this distance for HRP in Unit 11

Correlation distance

$$d_{ij} = \sqrt{\frac{1}{2}\left(1 - \rho_{ij}\right)}$$



ρ = 1 gives distance 0; ρ = −1 gives distance 1.

Cut the tree and sector-like groups appear.

DBSCAN defines clusters as dense regions

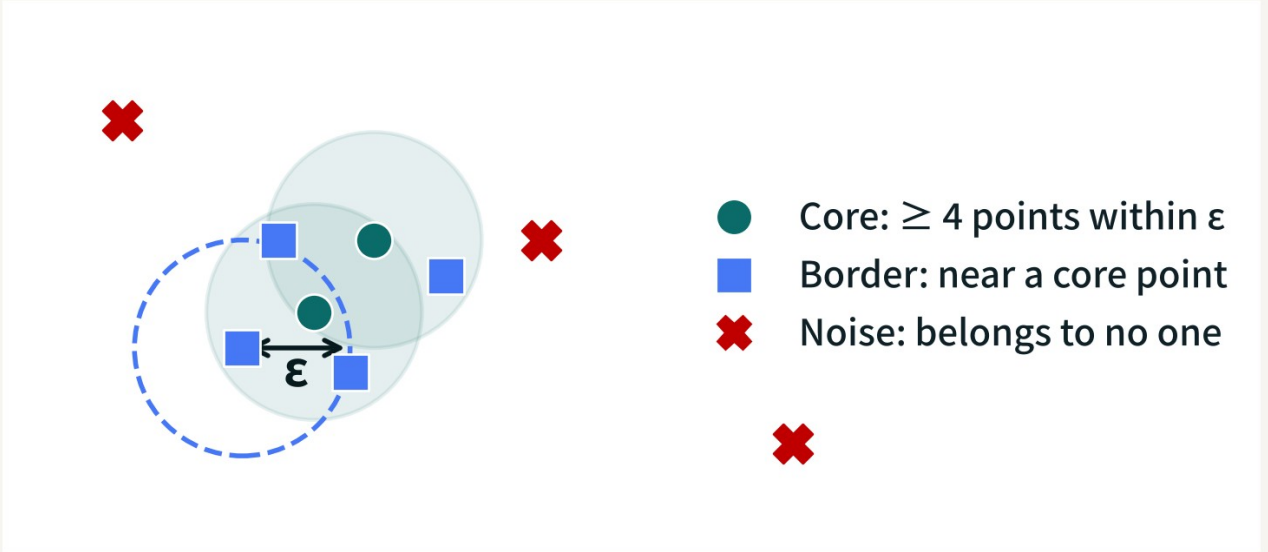
Density-Based Spatial Clustering of Applications with Noise (Ester et al., 1996)

ε-neighbourhood of point p

$$N_\varepsilon(p) = \{\, q : \|p - q\| \le \varepsilon \,\}$$

Core point rule

$$|N_\varepsilon(p)| \ge \text{minPts} \;\Rightarrow\; p \text{ is a core point}$$

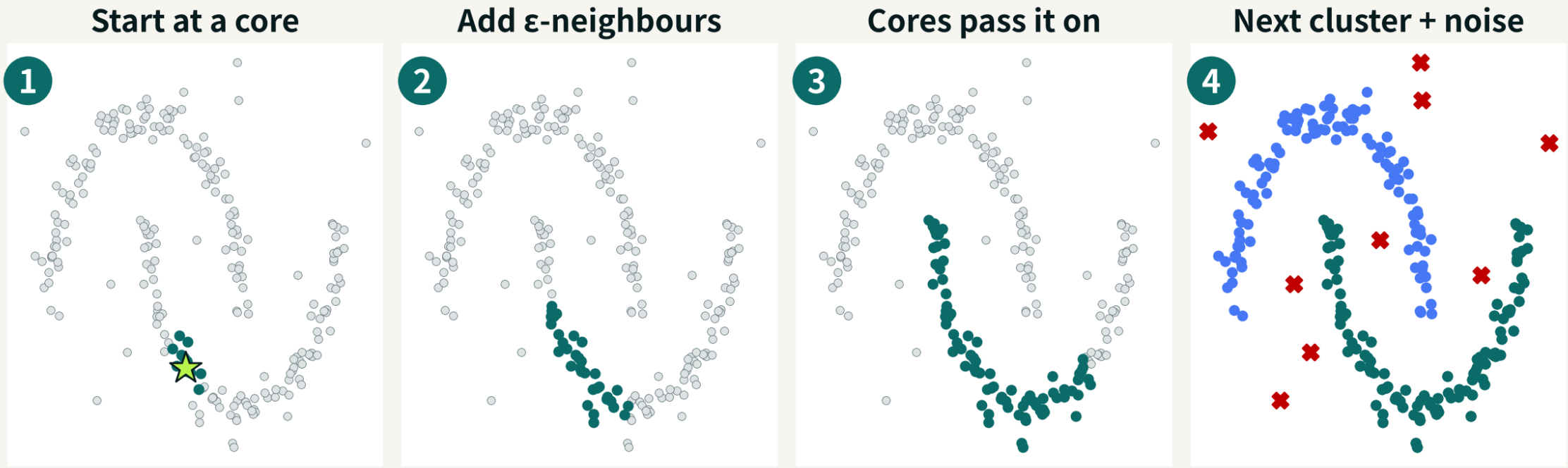


※ minPts = 4 (the point itself counts)

Two knobs, ε and minPts. No K to choose.

A DBSCAN cluster grows from core to core

Two interleaved half-moons plus scattered outliers; ε = 0.2, minPts = 5

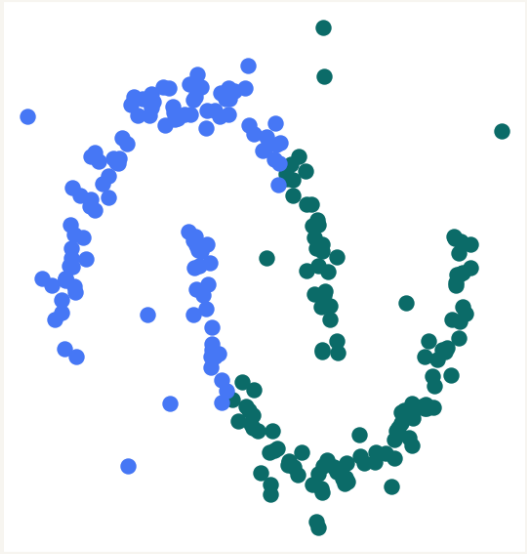


※ **Points reachable from no core point are labelled noise (−1). In finance: outlier days, odd trades.**

Each algorithm sees a different shape of cluster

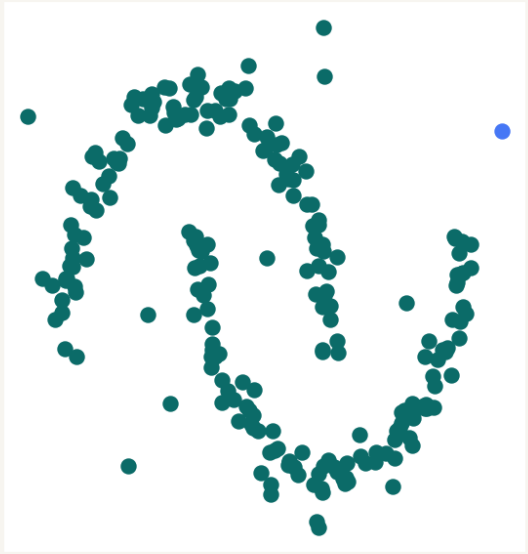
Same data, three algorithms. Colour = cluster, red X = noise

K-Means (K = 2)



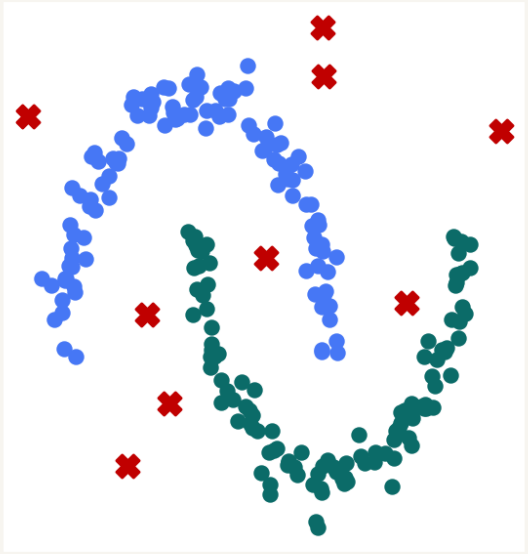
Cuts straight through the moons

Hierarchical (single linkage)



Follows the chain, but noise sticks

DBSCAN (ε = 0.2, minPts = 5)



Right shapes and noise flagged

Choose the clustering method by what you know in advance

Same goal, different assumptions about the shape and the number of groups

Method	Choose K?	Shape it finds	Outliers	Finance use
K-Means	**Yes**	**Round blobs**	**Forced in**	**Regimes, segments**
Hierarchical	**Cut later**	**Tree of groups**	**Forced in**	**Stock trees, HRP**
DBSCAN	**No (ε, minPts)**	**Any dense shape**	**Flagged**	**Odd days, fraud**

No method is right in general.

Check that the groups make economic sense.

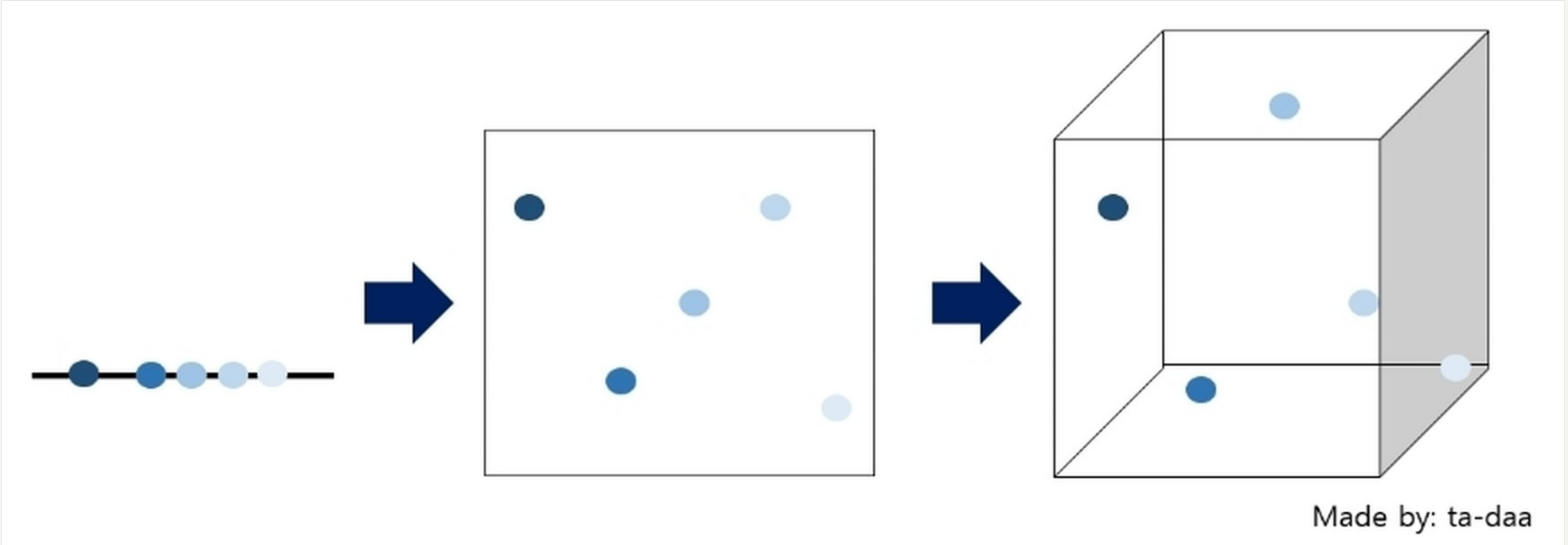
02

Shrinking Features

Curse of dimensionality · PCA · t-SNE

The same five points thin out as each dimension is added

Curse of dimensionality: line → square → cube, with the data unchanged



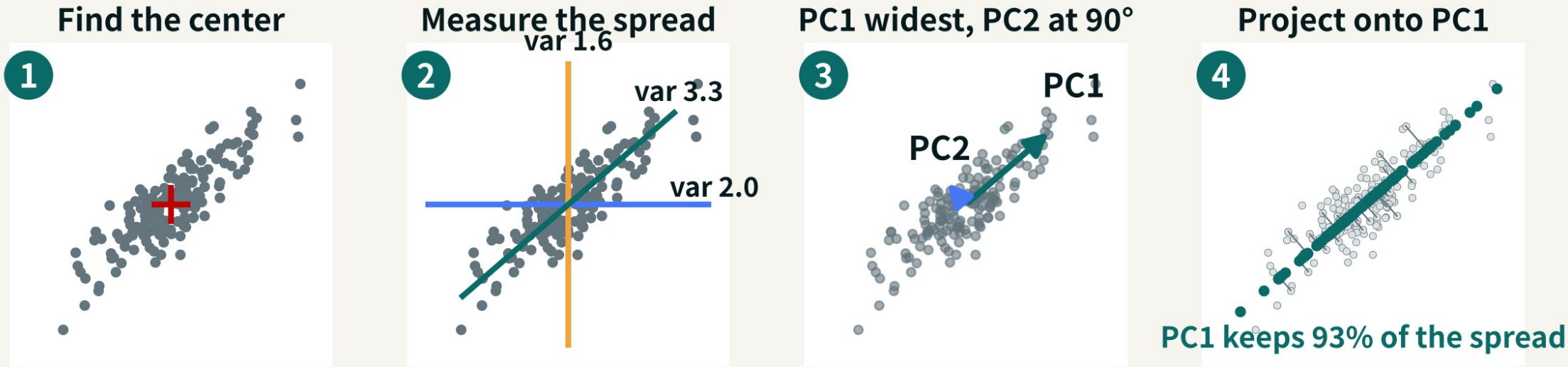
1-D: crowded 2-D: gaps 3-D: empty space

Every new feature multiplies the empty space, so distances lose meaning.

Clustering and KNN suffer; keep only the informative axes.

PCA rotates the axes toward the widest spread

Principal Component Analysis on two correlated features



※ Spread = variance. Each new axis is a weighted mix of the original features.

Correlation decides how much one component can carry

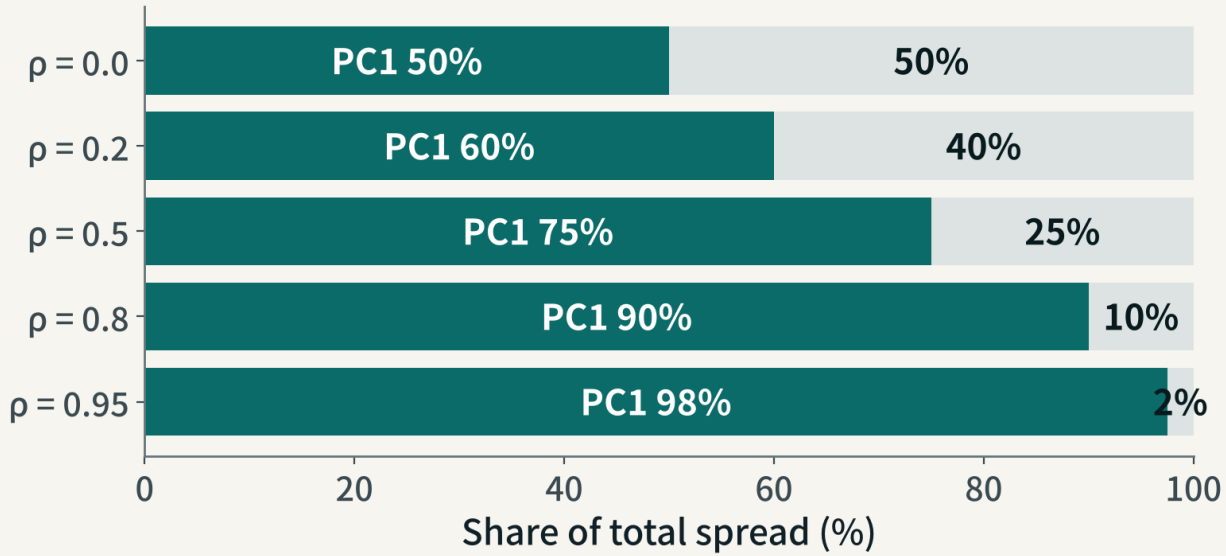
Eigenvector v = a direction; eigenvalue λ = the spread along it

PCA in one line

$$\Sigma v_k = \lambda_k v_k, \qquad \text{share}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

Two stocks with correlation ρ

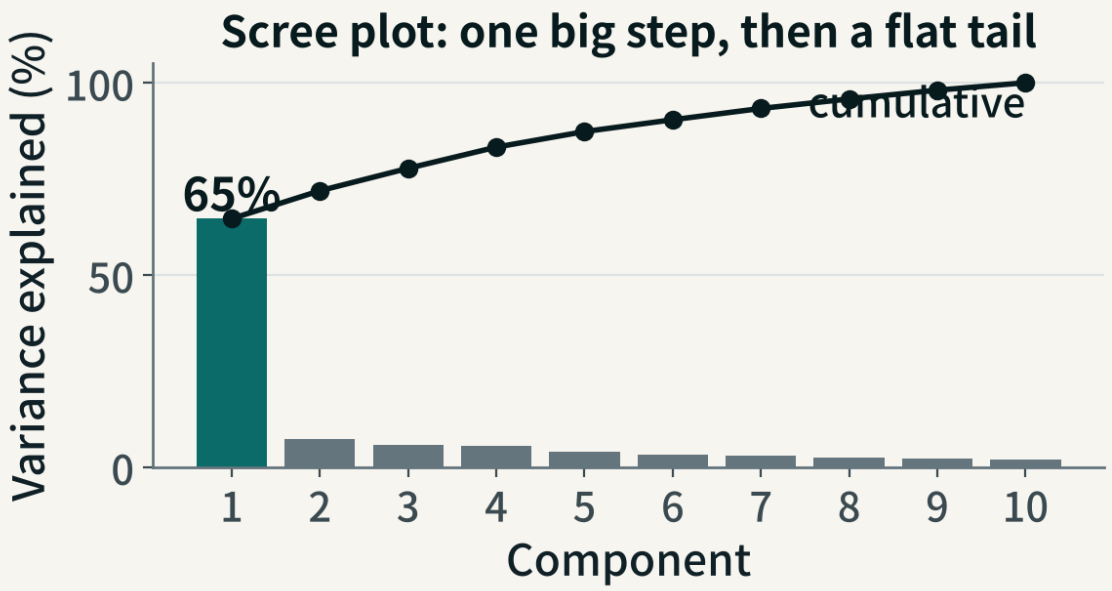
$$\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \Rightarrow \lambda_{1,2} = 1 \pm \rho$$

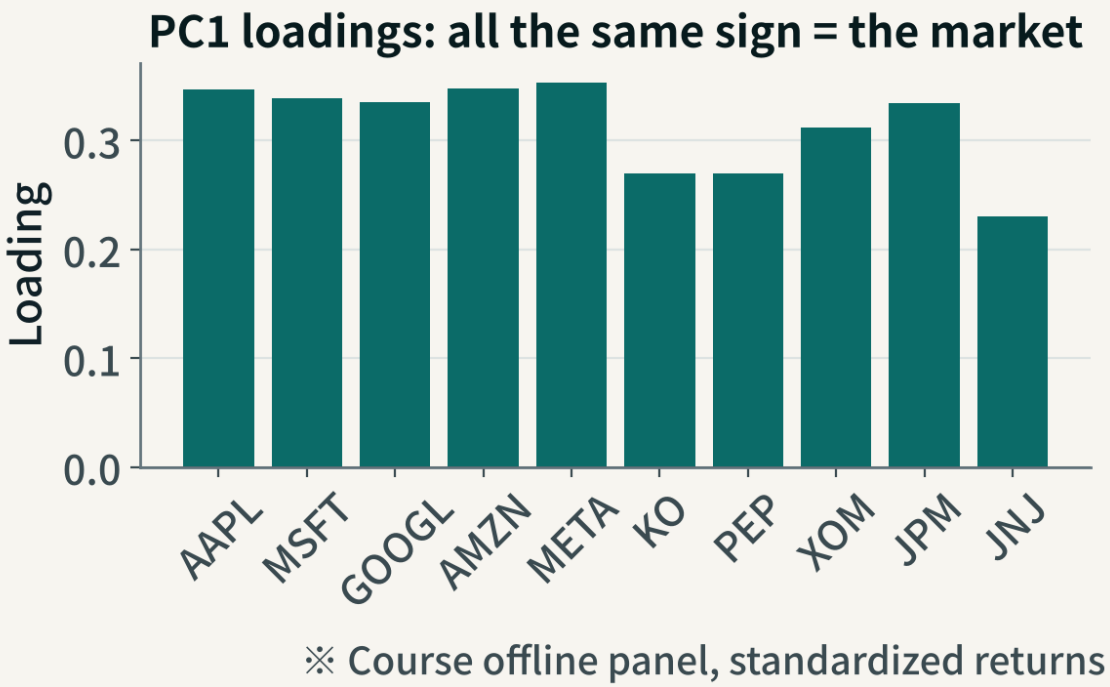


Compression pays only when features are correlated.

On stock returns the first component is the market

PCA on ten standardized daily return series



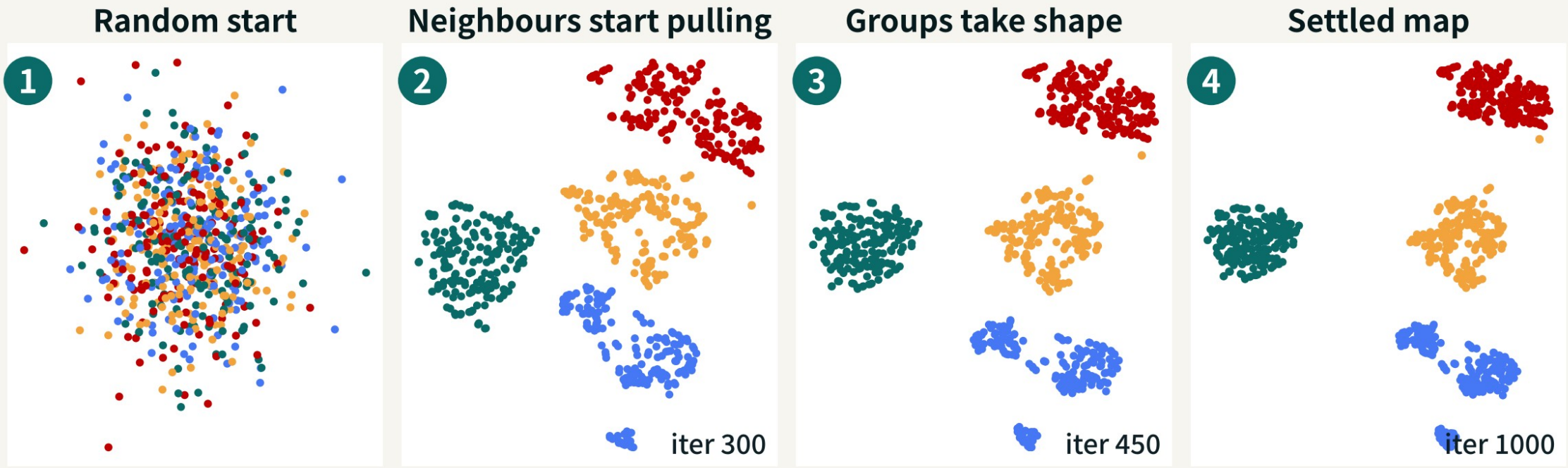


※ Course offline panel, standardized returns

※ PC1 explains about 65% here. Later components are sector tilts, then noise.

t-SNE pulls neighbours together on a 2-D map

t-distributed Stochastic Neighbor Embedding on 64-pixel handwritten digits (0, 1, 4, 7)



※ Colours come from the true digit and are used only to check the map. t-SNE never sees them.

t-SNE matches neighbour probabilities in two spaces

van der Maaten & Hinton (2008). x = original point, y = its position on the map

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

$$\mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

In plain words

① **p: who is close in the data**

② **q: who is close on the map**

③ **KL: penalty if they differ**

σ is set by perplexity

Perplexity is roughly the number of neighbours each point keeps.

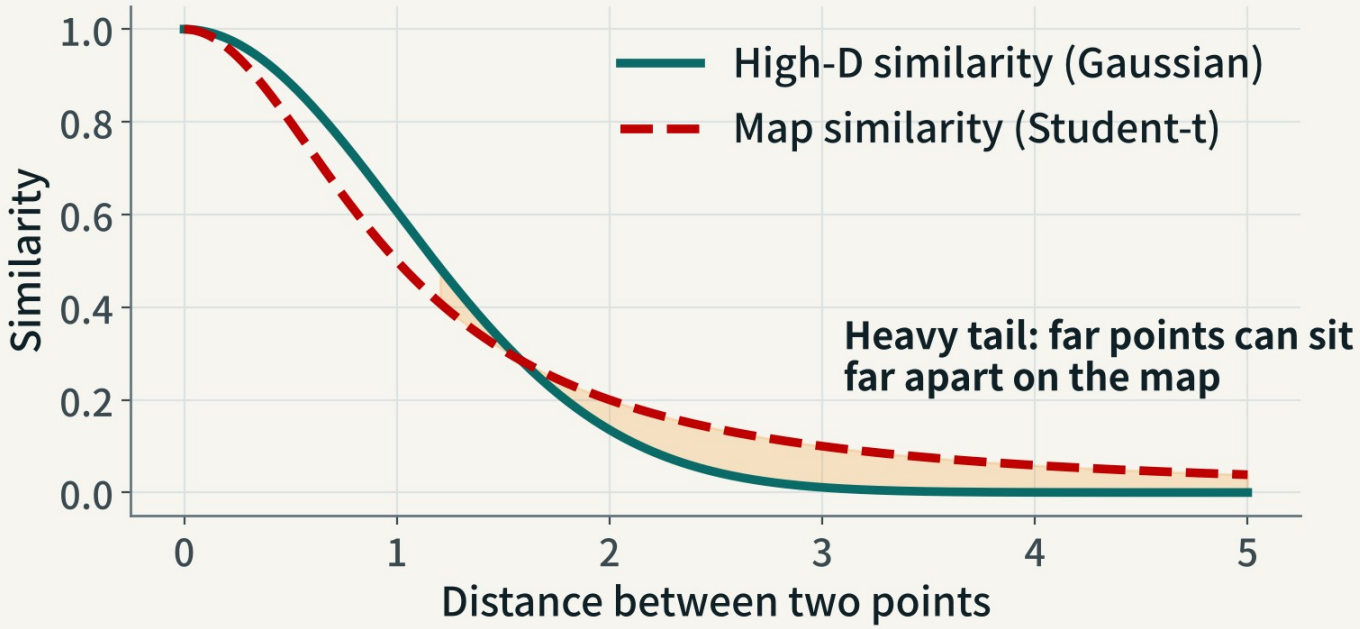
The heavy-tailed map kernel keeps clusters apart

Similarity as a function of distance in each space

Crowding problem

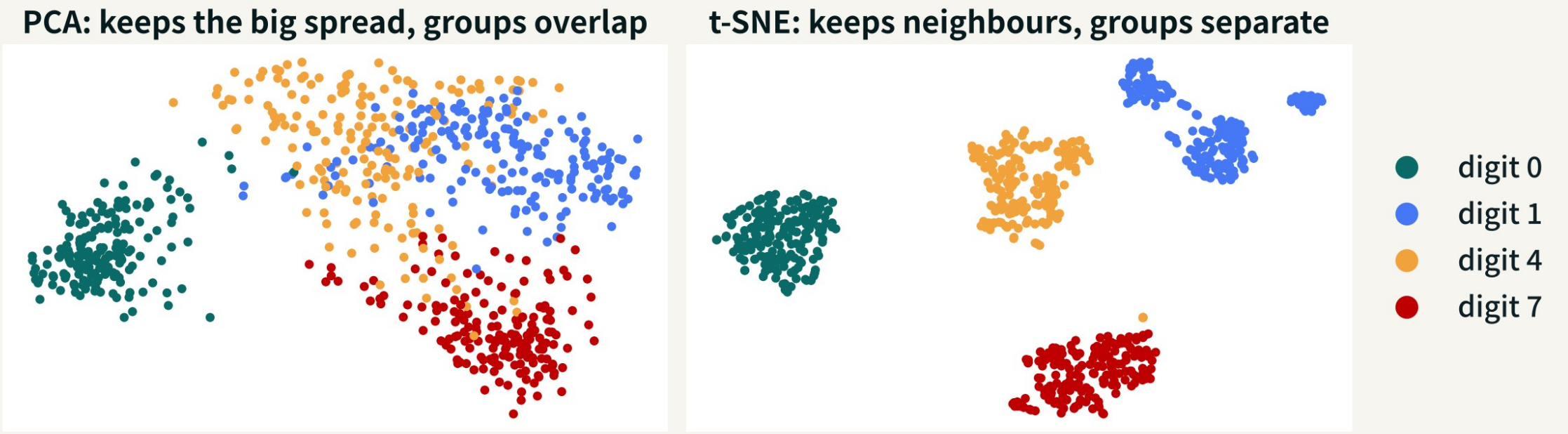
- 2-D has little room
- Gaussian crowds them
- Heavy tail frees them

Clean, separated islands



PCA keeps global spread while t-SNE keeps local neighbours

Same 64-dimensional digits squeezed to two dimensions

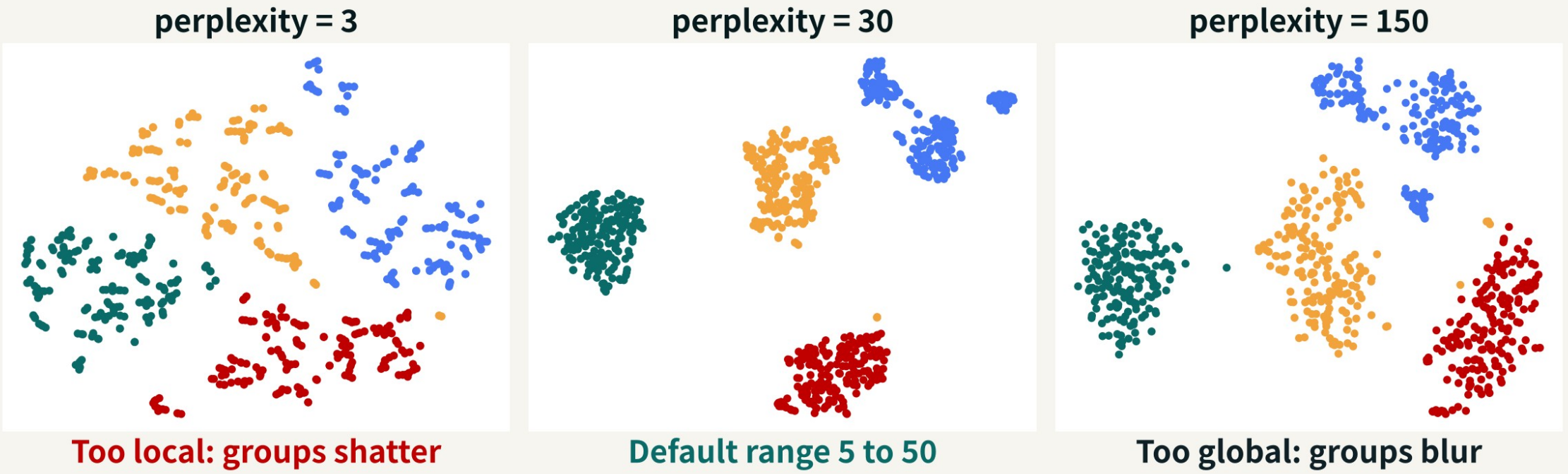


※ 64-pixel handwritten digits (scikit-learn), labels used only for colour

※ Use PCA to compress inputs for a model; use t-SNE to look at structure.

Perplexity sets how many neighbours each point considers

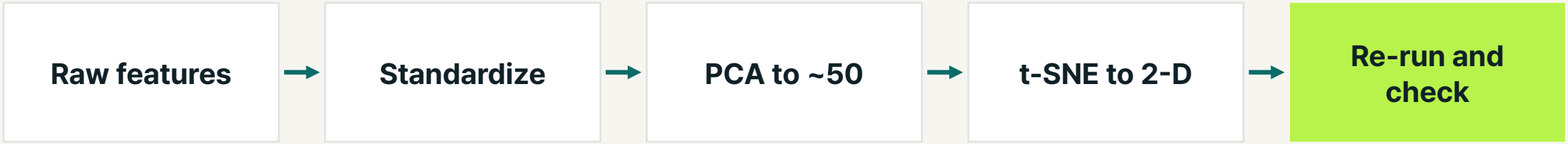
Same data, three perplexity values



※ On a t-SNE map, cluster sizes and gaps between clusters carry no meaning. Never feed it to a model.

In practice PCA compresses first and t-SNE maps second

The standard recipe when there are many features: t-SNE is slow and sensitive to noise directions



WHY STANDARDIZE

Equal units

No feature dominates

WHY PCA FIRST

Faster, cleaner

Drops noise directions

WHY RE-RUN

Shapes can move

Change seed, perplexity

Trust islands that survive every re-run.

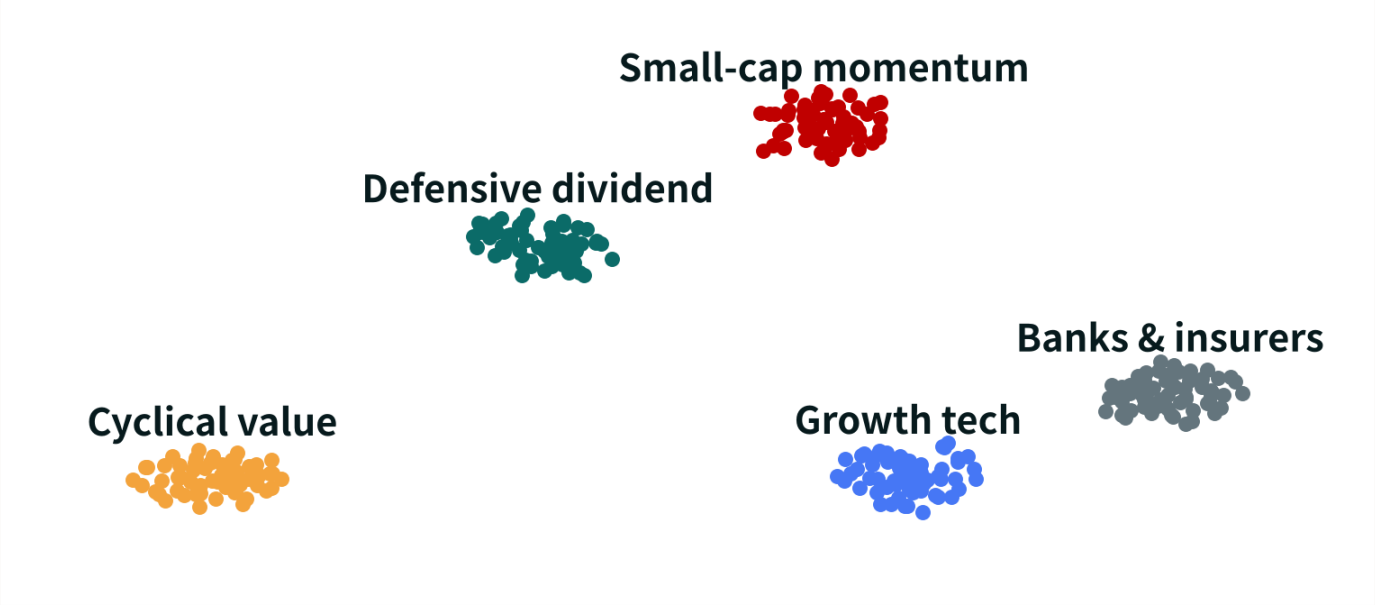
A map of stock characteristics shows style groups at a glance

Each stock described by 30 traits such as value, momentum, volatility and profitability

How to use it

- Islands = similar style
- Compare with sectors
- Confirm by clustering

Exploration, not proof



※ Simulated: 300 stocks × 30 characteristics → standardize → PCA 10 → t-SNE

03

Cleaning Noise

Estimation gap · Marchenko-Pastur · Denoising

A large correlation matrix has more unknowns than evidence

A typical institutional universe with five years of daily history

Stocks N	Correlation pairs	Trading days T	Days per pair
500	**~125,000**	**1,260**	**≈ 0.01**

Pairs to estimate

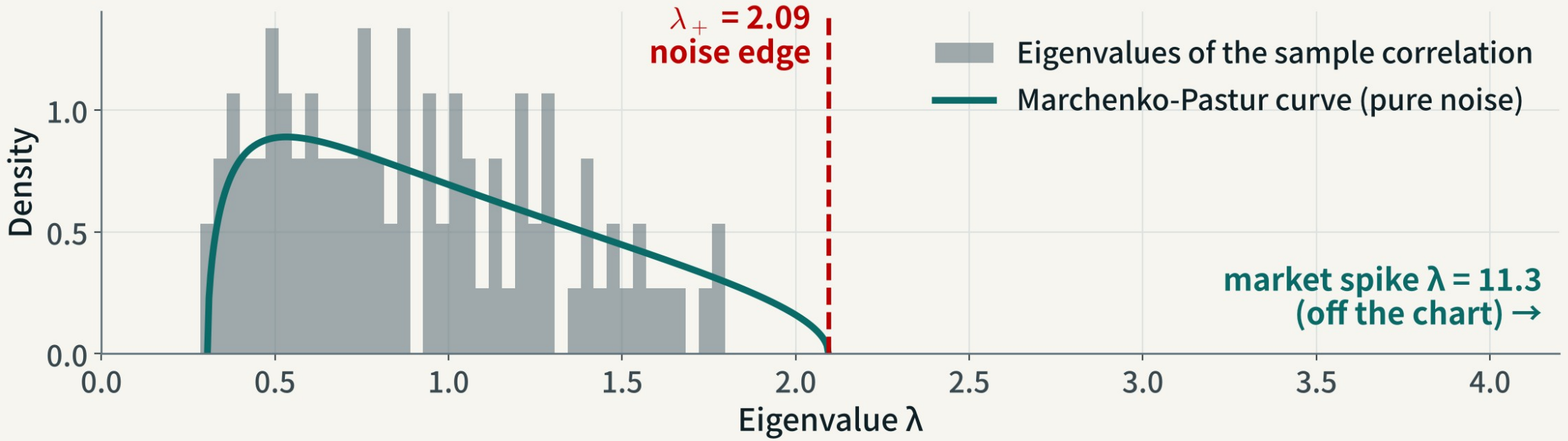
$$\frac{N(N-1)}{2} = \frac{500 \times 499}{2} \approx 125{,}000$$

Most measured correlations are mirages.

Optimizers chase them and build fragile portfolios.

Pure noise fills a predictable bulk; real factors spike out of it

Marchenko-Pastur law: the eigenvalue spread of a correlation matrix built from pure noise



※ Simulated: N = 100 stocks, T = 500 days, one true factor

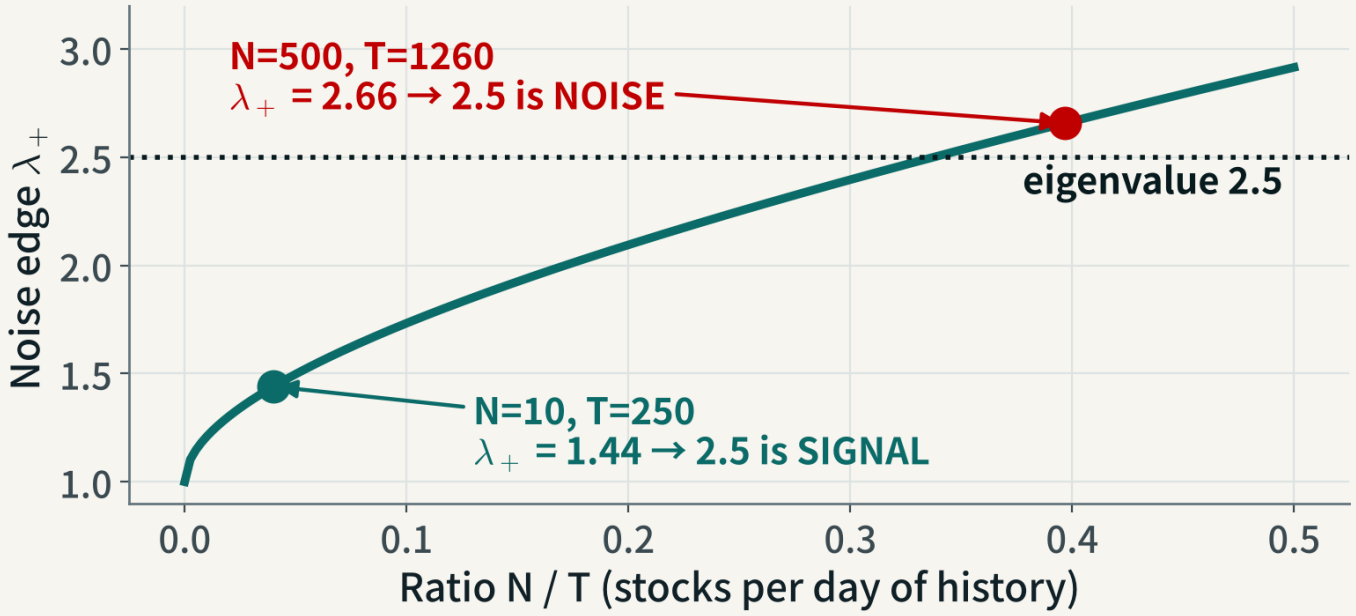
※ The bulk would appear even if the stocks were completely unrelated. Only the spike is structure.

The same eigenvalue can be signal or noise

N = number of stocks, T = number of days. Eigenvalues below the edge are noise

Marchenko-Pastur noise edge

$$\lambda_+ = \left(1 + \sqrt{N/T}\right)^2$$



More stocks or less history raise the edge.

2.5 is signal for 10 stocks but noise for 500.

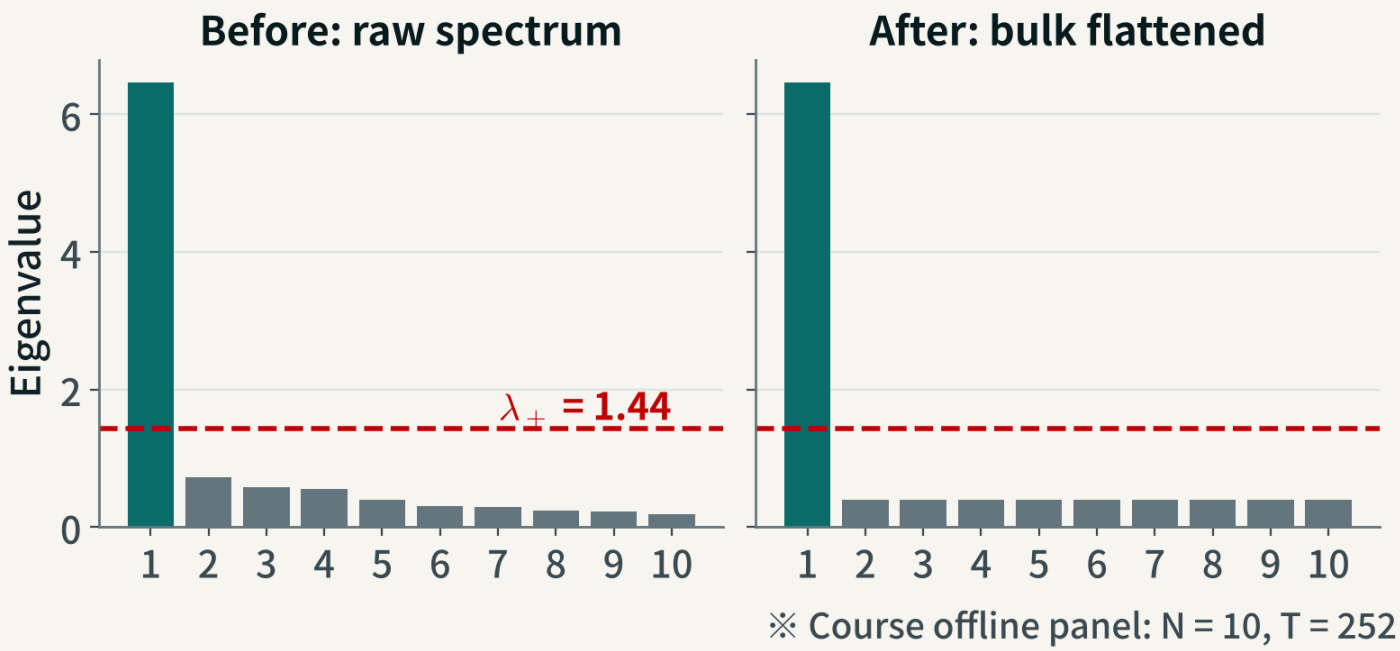
Denoising flattens the bulk and keeps the signal

López de Prado's three moves on the course panel

Recipe

- 1 Find all eigenvalues
- 2 Below edge → average
- 3 Rebuild the matrix

Optimize (Unit 11)



The minimum-variance portfolio puts the inverse covariance in charge

Global minimum-variance (GMV) portfolio: fully invested, lowest predicted risk (Markowitz)

GMV weights

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top\Sigma^{-1}\mathbf{1}}$$

SETUP

50 stocks, 120 days

1 market + 5 sectors

RAW

Sample covariance

As estimated

DENOISED

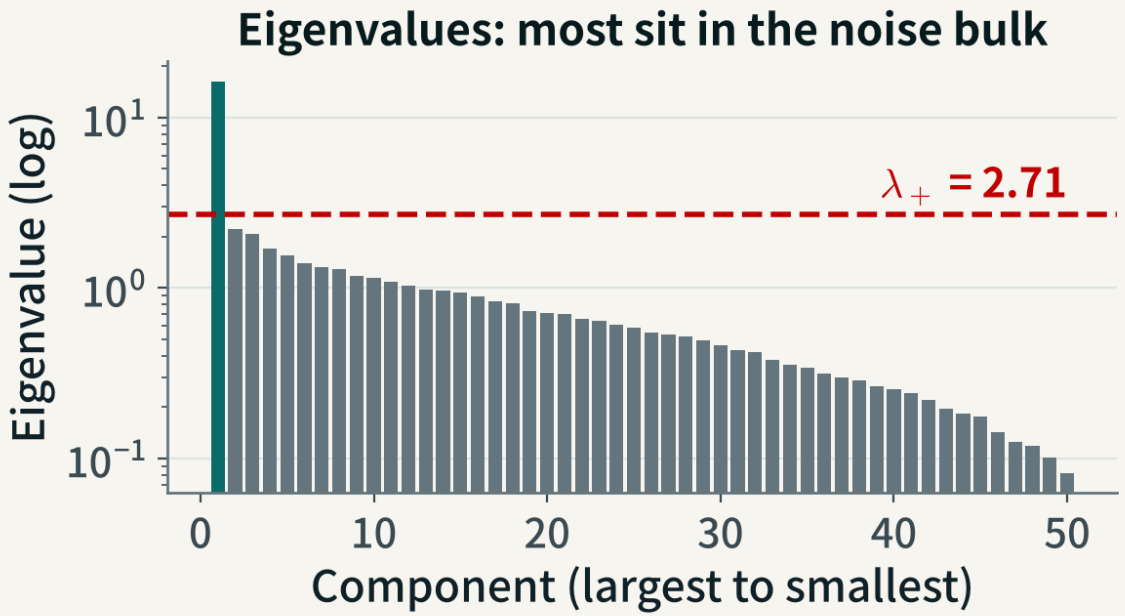
MP-cleaned

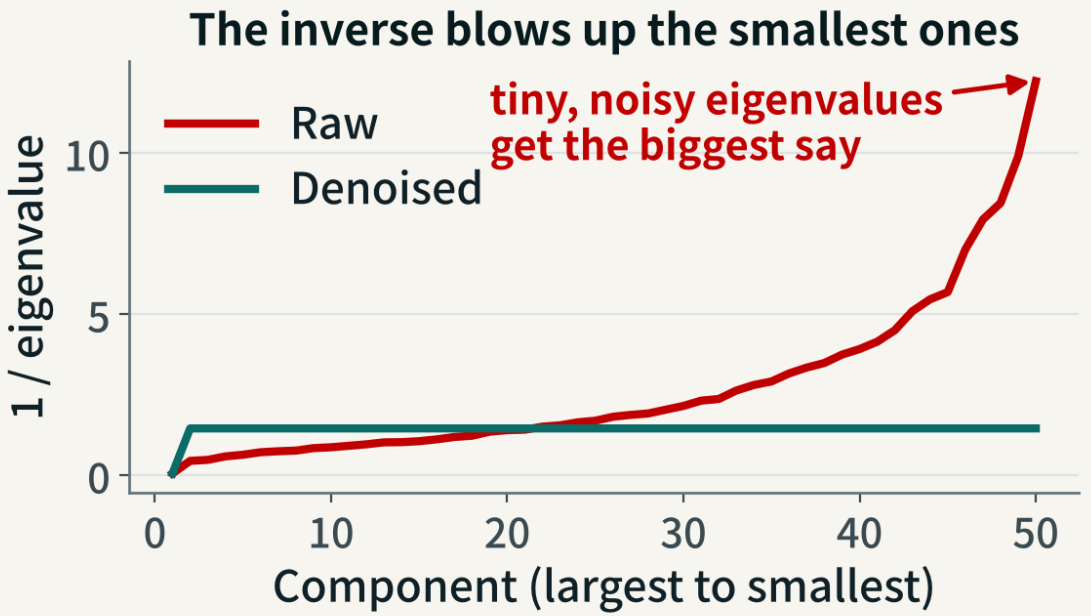
Same vols, clean corr

The inverse magnifies every estimation error in the matrix.

Inverting a noisy matrix turns tiny eigenvalues into big bets

Σ^{-1} divides by each eigenvalue, so the smallest, noisiest ones dominate the weights



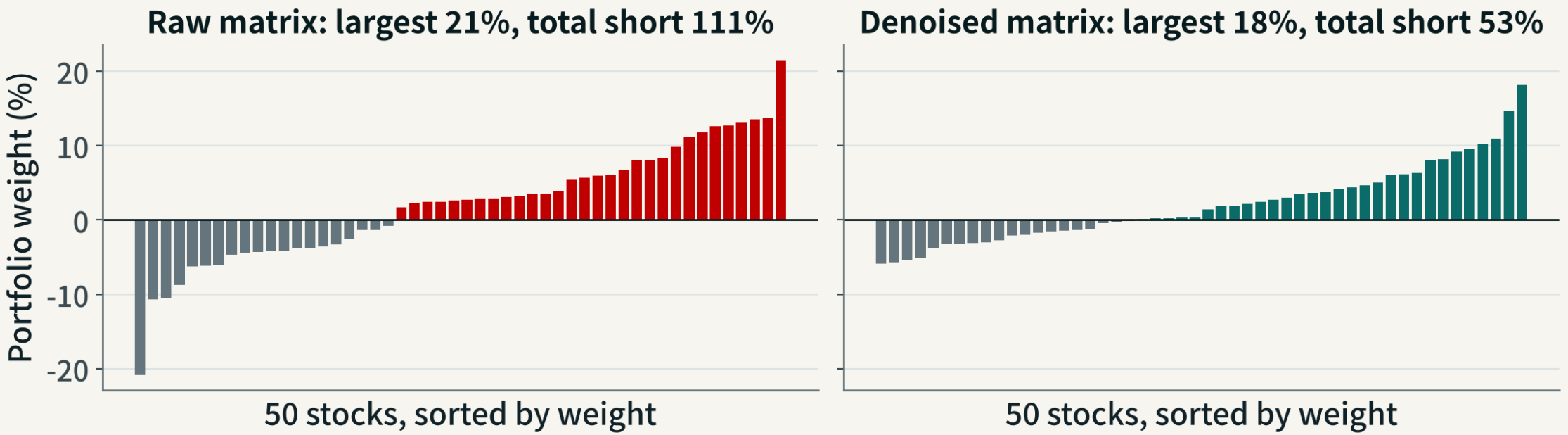


※ Simulated: 50 stocks, 120 days, 1 market + 5 sector factors

※ Denoising sets the bulk to its average, so no noise direction gets an outsized say.

Denoising tames extreme long and short positions

GMV weights from the same 120-day sample, raw versus denoised (a typical draw)



※ Minimum-variance weights from one 120-day sample; grey bars are short positions

※ **Raw weights bet on noise: they need more than 100% in shorts to exploit correlations that are not there.**

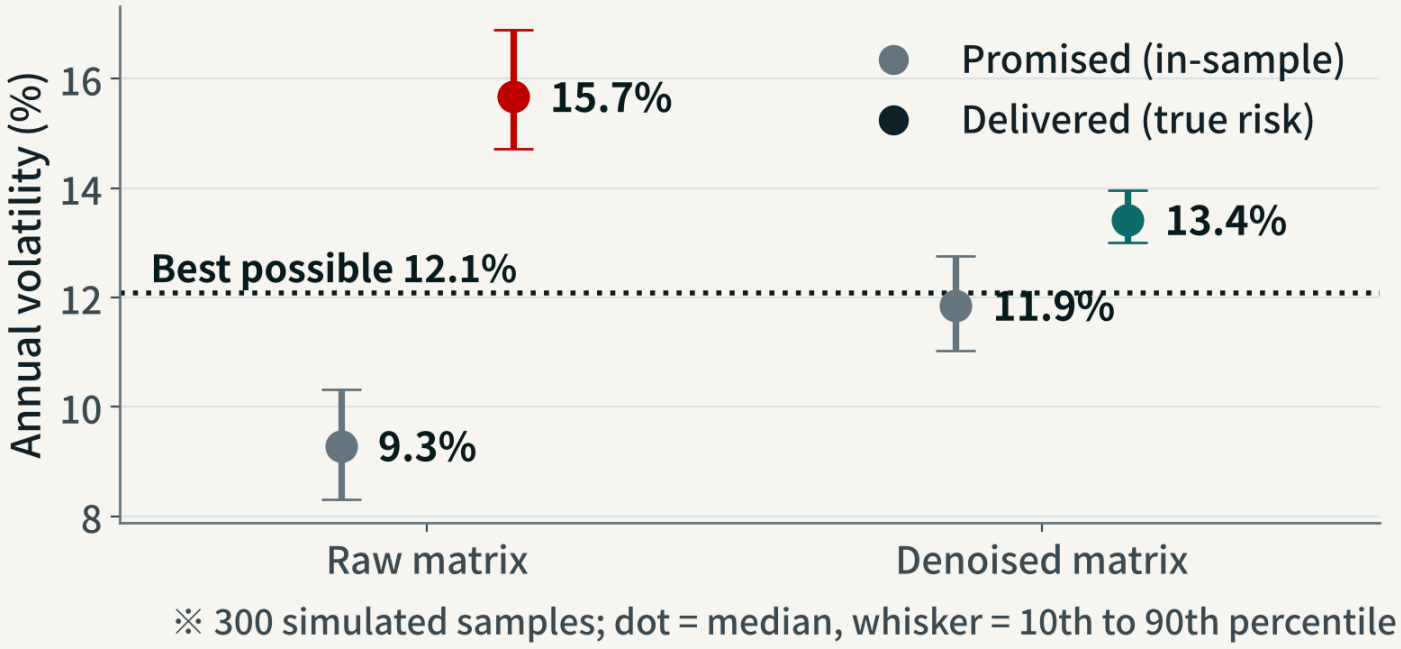
The denoised portfolio delivers lower risk and keeps its promise

Promised = risk the optimizer reports; delivered = true risk of those weights

Median of 300 runs

- Raw: 9.3 → 15.7%
- Denoised: 11.9 → 13.4%
- Best: 12.1%

Same data, honest risk



※ 300 simulated samples; dot = median, whisker = 10th to 90th percentile

Six tools cover the whole unsupervised toolbox

One line each: what it does and the one knob you must set

K-MEANS

Nearest center

Knob: K (elbow)

HIERARCHICAL

Merge tree

Knob: linkage, cut

DBSCAN

Dense regions

Knob: ε, minPts

PCA

Widest axes

Knob: no. of PCs

t-SNE

Neighbour map

Knob: perplexity

MP DENOISING

Flatten the bulk

Knob: N / T (given)

04

Build It with Claude Code

Three prompts reproduce every figure in this deck

Claude Code does the work while you check the result

Claude desktop app, Code tab. You paste a prompt and read the output

Open Code tab → **Paste prompt** → **Claude runs it** → **Read figures** → **Ask why**

CLAUDE DOES

Download, code, run

and fix its own errors

YOU DO

Paste and check

Does it make sense?

RULE

Explain every line

Ask why, not just do

Prompt 1 clusters the ten stocks three ways

Covers K-Means, the elbow, the dendrogram and DBSCAN

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	4 PNG figures	Real sectors?

Download daily returns for AAPL, MSFT, GOOGL, AMZN, META, KO, PEP, XOM, JPM and JNJ from 2020 to 2024 with yfinance. Cluster the 10 stocks three ways: (1) K-Means with K from 1 to 8 and an elbow plot, (2) hierarchical clustering on correlation distance with average linkage and a dendrogram, (3) DBSCAN, choosing eps from a k-distance plot. Save every figure as PNG and open them for me. Finish with three sentences: do the groups match real sectors, and which stock is the odd one out? If anything fails, fix it yourself and rerun.

Prompt 2 compresses returns with PCA and maps digits with t-SNE

Covers the scree plot, PC1 loadings, PCA vs t-SNE and perplexity

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	4 PNG figures	Market PC1?

Using the same 10-stock returns, standardize them and run PCA. Draw the scree plot with cumulative variance and a bar chart of PC1 loadings, and tell me whether PC1 behaves like a market factor. Then load scikit-learn's handwritten digits (0, 1, 4, 7 only) and draw PCA and t-SNE maps side by side, coloured by digit. Finally draw t-SNE at perplexity 3, 30 and 150. Save and open every figure, and explain each one in one plain sentence.

Prompt 3 denoises the correlation matrix before optimizing

Covers the eigenvalues, the Marchenko-Pastur edge and the before/after weights

WHERE	INPUT	OUTPUT	FEEDBACK
Code tab	This prompt	Table + 2 charts	Saner weights?

Compute the correlation matrix of the 10 stock returns and all its eigenvalues. Compute the Marchenko-Pastur edge using N stocks and T days, count how many eigenvalues are above it, replace the rest with their average, and rebuild the correlation matrix with ones on the diagonal. Plot the eigenvalues before and after with the edge as a dashed line. Print minimum-variance portfolio weights from the raw and the denoised matrix side by side, and explain in three sentences which weights look saner and why.

GROUP THE ROWS.

SQUEEZE THE COLUMNS.

CLEAN THE MATRIX.

Structure found without labels
must still make economic sense.